

UNIVERSITÀ
DEGLI STUDI
DI BERGAMO



UNIVERSITÀ DEGLI STUDI DI BERGAMO
Department of Management, Information and Production Engineering

Convex Optimization Analysis on the MAGIC Gamma Telescope Dataset

Optimization for Binary Classification

Amin Borqal, 1073928

Course: Optimization 2024/2025
Prof.ssa Maggioni

Outline

- Introduction
- Dataset
- Data Preprocessing
- Problem Setting
- Gradient Descent
- Stochastic & Adaptive Methods
- Result Analysis
- Newton's Method

Aim of This Work

The aim of this study is to analyze the **convergence** of optimization algorithms applied to a real dataset, in the context of convex, smooth, and Lipschitz functions.

The task is a **binary classification problem** solved via **logistic regression**.

MAGIC Gamma Telescope: What Is It?

The **MAGIC** (Major Atmospheric Gamma Imaging Cherenkov) telescope is located at La Palma, Canary Islands, at 2,200 m altitude. Its 17-metre mirror records the faint **Cherenkov light** emitted when cosmic particles enter the atmosphere at near-light speed.

The classification problem

Two types of showers reach the detector:

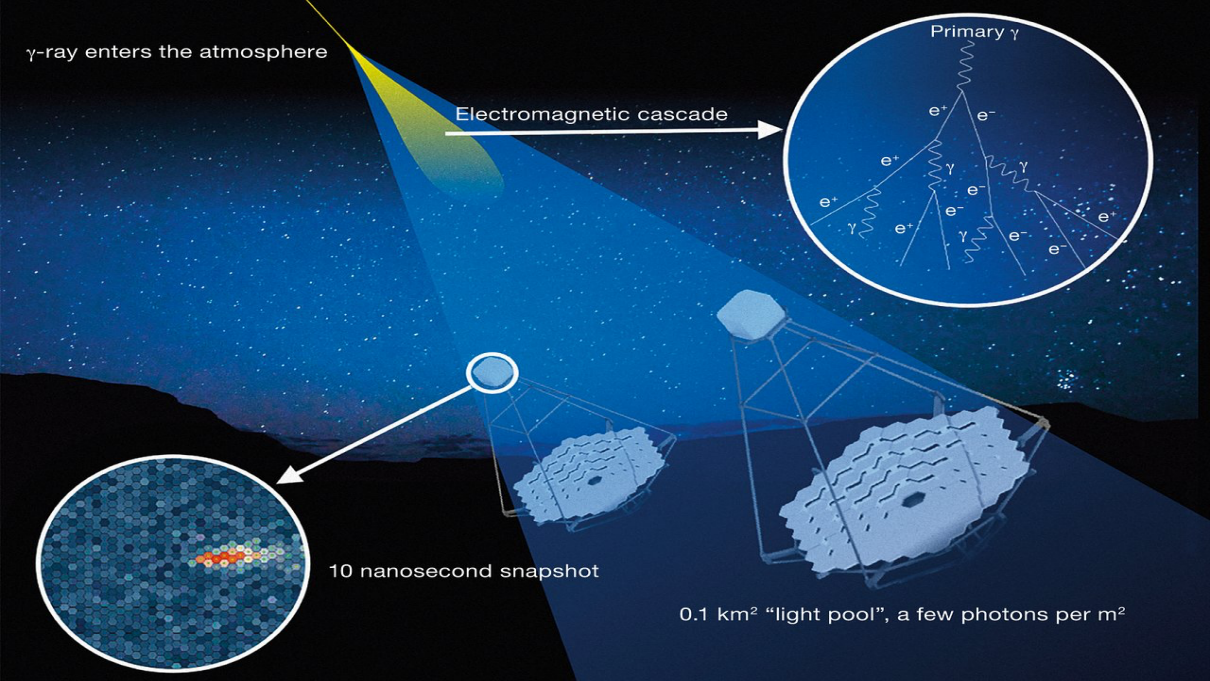
- **Gamma rays** (signal) — photons from astrophysical sources; compact, symmetric shower images.
- **Hadrons** (background) — cosmic-ray protons; wider, irregular shower images.

Goal

Discriminate **gamma-ray events** from the dominant hadron background using shower-image shape parameters (*Hillas parameters*).

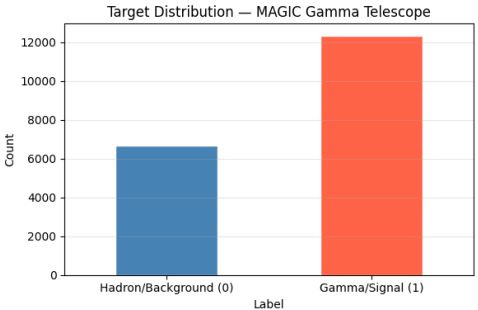
Task

Binary classification: **gamma** (signal, label 1) vs **hadron** (background, label 0).


γ-ray enters the atmosphere
Electromagnetic cascade
Primary γ
e+
e−
γ
10 nanosecond snapshot
0.1 km² "light pool", a few photons per m²

Dataset Summary

Property	Value
Observations	19,020
Features	10 Hillas parameters
Missing values	none
Task	Binary classification
Gamma (signal)	12,332 (64.8%)
Hadron (bkg)	6,688 (35.2%)
After preprocessing	**9 features**



Feature Description (1/2)

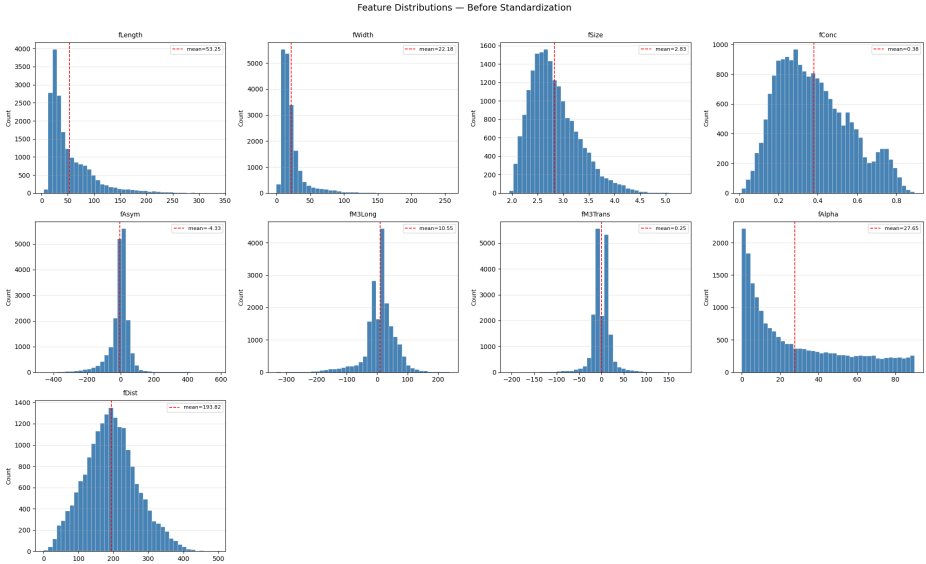
All 10 features are **Hillas parameters** — shape descriptors of the ellipse fitted to the Cherenkov shower image recorded by the telescope camera.

Feature	Physical meaning	Unit
fLength	Major axis of shower ellipse (elongation)	mm
fWidth	Minor axis of shower ellipse (lateral spread)	mm
fSize	$\log_{10}$ of total Cherenkov light collected	photons
fConc	Ratio of 2 brightest pixels / fSize (compactness)	–
fConc1	Ratio of brightest pixel / fSize	–

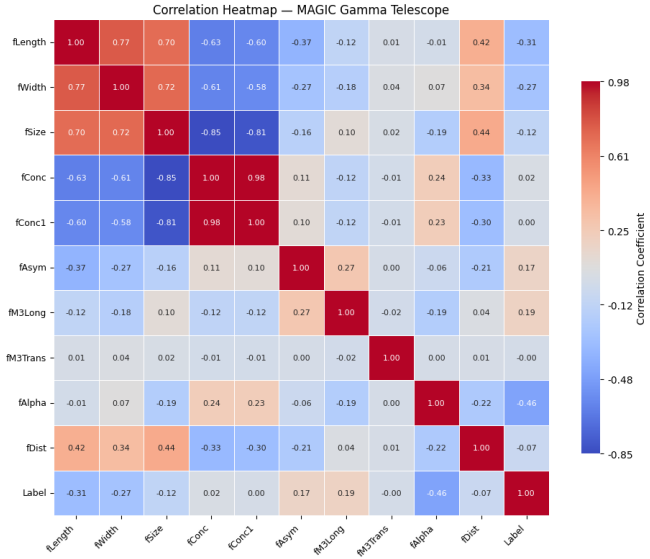
Feature Description (2/2)

Feature	Physical meaning	Unit
fAsym	Distance from brightest pixel to ellipse centre	mm
fM3Long	3rd moment along the major axis (skewness)	mm
fM3Trans	3rd moment along the minor axis	mm
fAlpha	Angle of major axis with direction to source	deg
fDist	Distance from camera centre to ellipse centre	mm

Feature Distributions: Before Standardization



Correlation Heatmap — All 10 Features

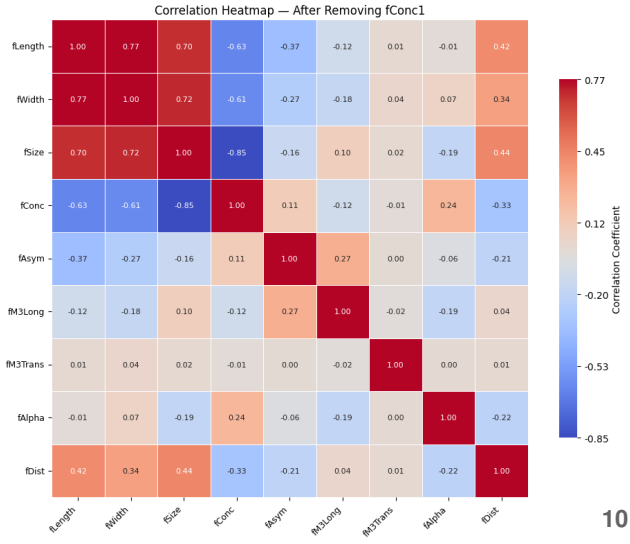


Feature Elimination: Dropping `fConc1`

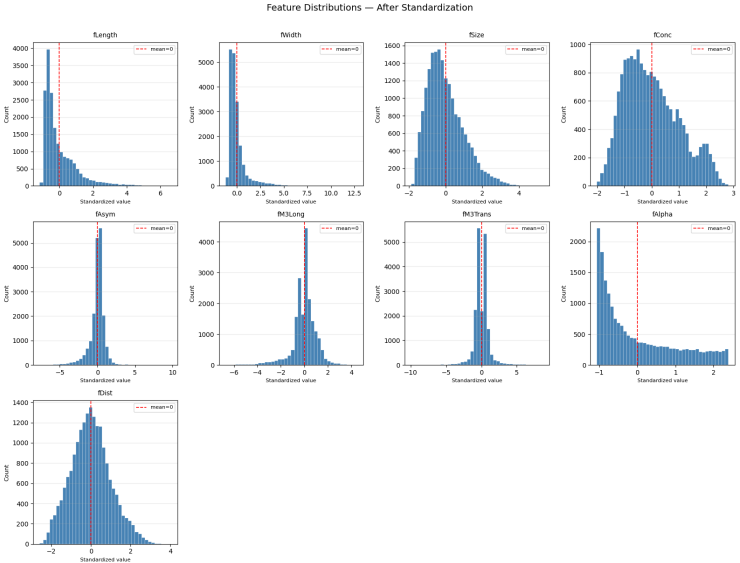
fConc and **fConc1** both measure the concentration of Cherenkov light in the brightest pixels:

- **fConc**: ratio using the *2 brightest* pixels
- **fConc1**: ratio using only the *brightest* pixel

Their Pearson correlation is $r = 0.98$, so they carry nearly identical information.



Feature Distributions: After Standardization



Logistic Regression: Problem Setting

The output $y_i \in \{0, 1\}$ is a **Bernoulli** random variable with probability mass function:

$$P(y_i \mid a_i, w) = \hat{y}_i^{y_i}(1 - \hat{y}_i)^{1-y_i}$$

where the **logistic (sigmoid) function** models the predicted probability:

$$\hat{y}_i = \sigma(a_i^\top w) = \frac{1}{1 + e^{-a_i^\top w}}$$

Coefficient estimation

We want to find w^* such that $\hat{y}_i \approx y_i$ for all observations. This is achieved by **Maximum Likelihood Estimation (MLE)**.

Logistic Regression: Likelihood

Since observations are i.i.d., the joint likelihood factorises:

$$L(w \mid A, y) = \prod_{i=1}^{n} \hat{y}_i^{\,y_i} (1 - \hat{y}_i)^{1-y_i}$$

Taking the logarithm gives the **log-likelihood**:

$$\ell(w) = \sum_{i=1}^{n} \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

From MLE to minimization

Maximising $\ell(w)$ is equivalent to **minimizing** the **negative log-likelihood** (binary cross-entropy):

$$\min_w \; f(w) = -\frac{1}{n} \ell(w)$$

Logistic Regression: Optimization problem

Objective: Binary Cross-Entropy

$$\min_w \; f(w) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]$$

Gradient

$$\nabla f(w) = \frac{1}{n} A^\top (\hat{y} - y)$$

Hessian

$$\nabla^2 f(w) = \frac{1}{n} A^\top W A$$

$$W = \mathrm{diag}(\hat{y}_i(1-\hat{y}_i))$$

Convexity

$W \succ 0 \;\Rightarrow\; \nabla^2 f(w) \succeq 0$: **strictly convex** cost $\Rightarrow$ unique global minimum, gradient descent / Newton's method guaranteed to converge.

Logistic Regression: Convexity & Lipschitz

Strict Convexity

$\nabla^2 f = \frac{1}{n} A^\top W A$ is **positive definite** since W is diagonal with entries:

$$\hat{y}_i(1 - \hat{y}_i) \in (0, 1)$$

$$\Rightarrow f \text{ is strictly convex}$$

Lipschitz Gradient

$\hat{y} \in (0,1)^n$ bounded $\Rightarrow$ $\|\nabla f\|$ bounded:

$$\|\nabla f(w)\| \leq B$$

$$B = \frac{\|A^\top A\| R + \|A^\top y\|}{n}$$

$R \geq \|w^*\|$: empirically assumed

$R = 25$.

Logistic Regression: Smoothness & Strong Convexity

L-Smoothness

Bounded Hessian:

$$0 < A^\top W A \leq A^\top A$$

$$L = \frac{\|A^\top A\|_2}{4n}$$

Sufficient decrease ($\gamma = 1/L$):

$$f(w_{t+1}) \leq f(w_t) - \frac{1}{2L}\|\nabla f(w_t)\|^2$$

Strong Convexity (μ)

$$\mu = \frac{\lambda_{\min}(A^\top A)}{4n}$$

$$\kappa = \frac{L}{\mu}$$

Small $\kappa \Rightarrow$ faster convergence of gradient descent.

Summary

f is μ-strongly convex and L-smooth $\Rightarrow$ GD with $\gamma = 1/L$ converges at linear rate $\mathcal{O}\big((1 - \mu/L)^t\big)$.

Constants choice

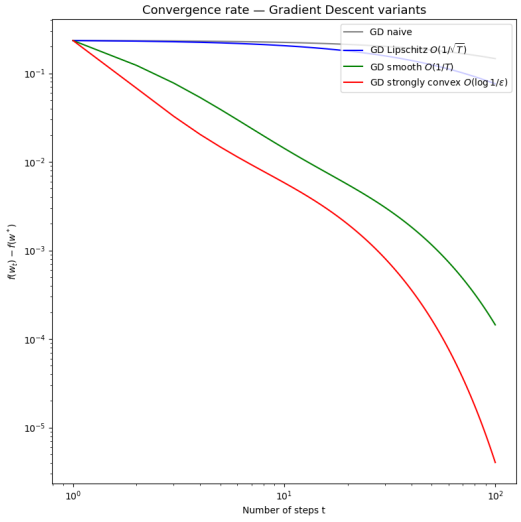
Constant	Formula	Value
L	$\|A^\top A\|_2/(4n)$	0.8769
μ	$\lambda_{\min}(A^\top A)/(4n)$	0.0311
$\kappa = L/\mu$	condition number	28.24
R	$\|w^*\|$ upper bound (empirical)	25
$f(w^*)$	optimal value	0.4587

Gradient Descent: Update Rule

$$w_{t+1} = w_t - \gamma \nabla f(w_t)$$

Variant	Step size γ	Rate	Assumption
Naive	0.01 (arbitrary)	no guarantee	none
Bounded grad.	$R/(B\sqrt{T})$	$\mathcal{O}(1/\sqrt{T})$	Lipschitz, $\|\nabla f\| \leq B$
L-Smooth	$1/L$	$\mathcal{O}(1/T)$	L-smooth
Strongly convex	$2/(L+\mu)$	$\mathcal{O}(\rho^T)$	L-smooth $+\mu$-s.c.

GD: Results ($T = 100$ iterations)



Variant	$f(w_T) - f(w^*)$	Rate
Naive	1.47×10^{-1}	—
Bounded	7.61×10^{-2}	$\mathcal{O}(1/\sqrt{T})$
L-Smooth	1.44×10^{-4}	$\mathcal{O}(1/T)$
Str. convex	$\mathbf{4.03 \times 10^{-6}}$	$\mathcal{O}(\rho^T)$

Gradient Descent: Experimental Results

Method	Time (ms)	Final gap $f(w_T) - f(w^*)$
GD naive	87 ms	1.47×10^{-1}
GD bounded (Lipschitz)	315 ms	7.61×10^{-2}
GD smooth $(1/L)$	992 ms	1.44×10^{-4}
GD str. convex $\left(2/(L+\mu)\right)$	202 ms	4.03×10^{-6}

Conclusion

Leveraging problem structure (smoothness, strong convexity) drastically reduces the final gap with a close runtime.

Stochastic GD: Objective

$$f(w) = \frac{1}{n} \sum_{i=1}^{n} f_i(w)$$

where $f_i(w) = -\big[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)\big]$.

Instead of using the full gradient on all $n = 19{,}020$ samples, SGD approximates it on a random **mini-batch** of size $m = 64$ at each step — making each iteration roughly **300**$\times$ cheaper.

Plus two **adaptive** methods that automatically tune the learning rate: AdaGrad and Adam.

Mini-Batch SGD: Algorithm & Variants

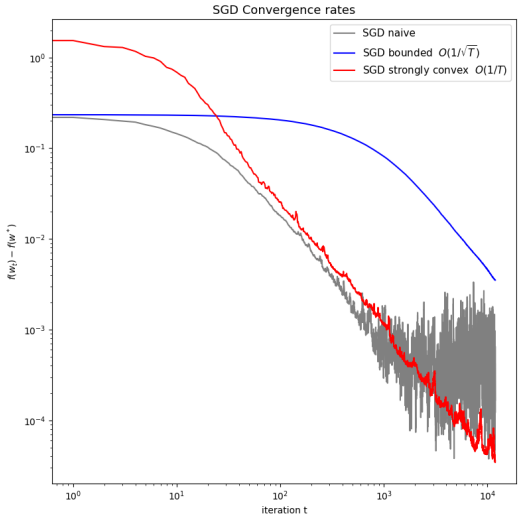
At each iteration, sample a mini-batch $\mathcal{B}_t \subseteq \{1, \ldots, n\}$, $|\mathcal{B}_t| = m$:

$$w_{t+1} = w_t - \gamma_t \cdot \frac{1}{m} \sum_{j \in \mathcal{B}_t} \nabla f_j(w_t)$$

Variant	**Step size** γ_t	**Rate**
SGD Naive	0.1 (fixed)	no guarantee
SGD Bounded	$R/(B\sqrt{T})$	$\mathcal{O}(1/\sqrt{T})$
SGD Str. Convex	$\gamma_0/(t+1)$, $\gamma_0 = 1/\mu$	$\mathcal{O}(1/T)$

Mini-Batch size: $m = 64$ — **Epochs**: 40

SGD Variants: Convergence



Variant	$f(w_T) - f(w^*)$	Rate
Naive	1.67×10^{-4}	—
Bounded	3.51×10^{-3}	$\mathcal{O}(1/\sqrt{T})$
Str. Convex	$\mathbf{3.6 \times 10^{-5}}$	$\mathcal{O}(1/T)$

Note

The decreasing schedule $\gamma_t = 1/(\mu(t+1))$ achieves $\mathcal{O}(1/T)$, strictly better than $\mathcal{O}(1/\sqrt{T})$ from the bounded variant. The naive fixed $\gamma = 0.1$ works well here despite having no guarantee.

Adaptive Methods: AdaGrad & Adam

AdaGrad

Adapts the learning rate **per coordinate**:

$$[G_t]_i = \sum_{s=0}^{t} [\tilde{g}_s]_i^2$$

$$[w_{t+1}]_i = [w_t]_i - \frac{\gamma}{\sqrt{[G_t]_i + \epsilon}} [\tilde{g}_t]_i$$

Pros: suited for sparse/variable-magnitude features.

Cons: accumulated G_t can shrink step sizes to zero.

Adam (β_1=0.9, β_2=0.999, ϵ=10^{-8})

Combines **momentum** and **adaptive rates**:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\tilde{g}_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\tilde{g}_t^2$$

$$w_{t+1} = w_t - \frac{\gamma}{\sqrt{v_t} + \epsilon} m_t$$

Fast early convergence in practice; final gap 3.35×10^{-4}.

Newton's Method: Update Rule

$$w_{t+1} = w_t - \left[\nabla^2 f(w_t)\right]^{-1} \nabla f(w_t)$$

Hessian for Logistic Regression

$$\nabla^2 f(w) = \frac{1}{n} A^\top W A$$

$$W = \operatorname{diag}(\hat{y}_i(1 - \hat{y}_i))$$

Implementation

Solve the linear system at each step:

$$\nabla^2 f(w_t)\ \delta = \nabla f(w_t)$$

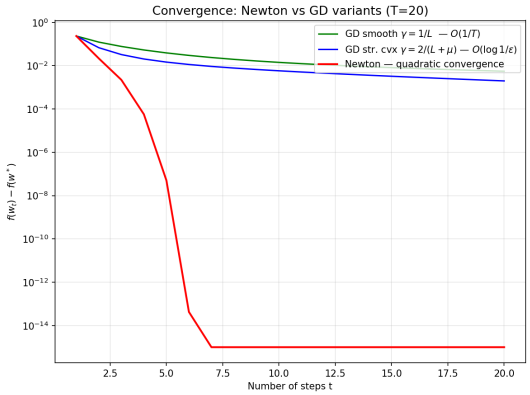
$$w_{t+1} = w_t - \delta$$

No matrix inversion: `np.linalg.solve` ($d \times d$).

Convergence

Quadratic convergence near w^*: error squares at each step. Reaches machine precision ($\sim 10^{-15}$) in just **20 iterations**.

Newton vs GD: Convergence ($T = 20$)



Method	Gap	Time
GD smooth	5.55×10^{-3}	10 ms
GD str. cvx	1.98×10^{-3}	19 ms
Newton	$\approx \mathbf{10^{-15}}$	**37 ms**

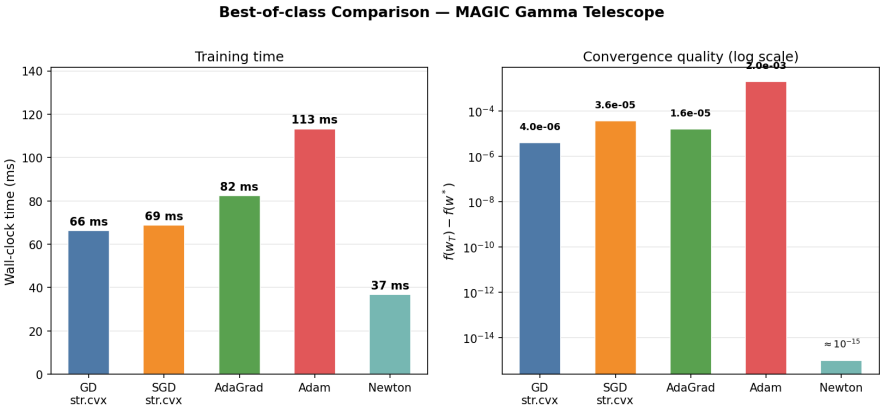
All methods run for $T = 20$ iterations.

All Methods: Final Comparison

Algorithm	$f(w_T) - f(w^*)$	**ms**
GD Naive	1.47×10^{-1}	87
GD Bounded	7.61×10^{-2}	315
GD Smooth	1.44×10^{-4}	992
GD Str. Cvx	4.03×10^{-6}	202
SGD Naive	1.67×10^{-4}	248
SGD Bounded	3.51×10^{-3}	388
SGD Str. Cvx	3.6×10^{-5}	183
AdaGrad	$\mathbf{7.42 \times 10^{-6}}$	271
Adam	3.35×10^{-4}	371
Newton	$\approx \mathbf{10^{-15}}$	**37**

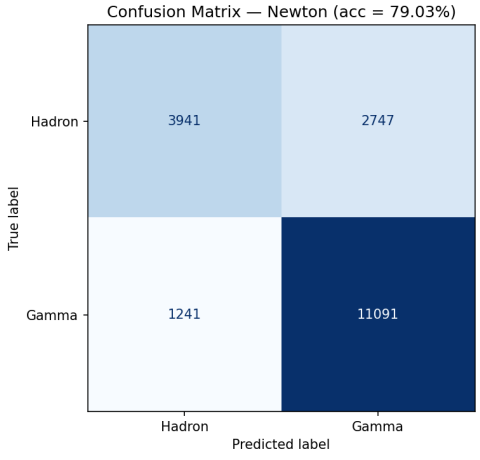
$$f(w^*) = 0.4587$$

Timing & Convergence



GD str. cvx & SGD str. cvx: $T=100$ | AdaGrad & Adam: 40 epochs | Newton: $T=20$

Confusion Matrix: Newton's Method



Binary Classification

$$\hat{y} = \mathbf{1}\left[\sigma(a^\top w) \geq 0.5\right]$$

- **Gamma** (signal) well-identified
- Some **Hadron** misclassified as Gamma (class imbalance)

Result (Newton, 20 iters)

Accuracy	**79.03%**
Gamma rec.	90%
Hadron rec.	59%
Gap $f(w_T) - f(w^*)$	$\approx 10^{-15}$

References

- R. K. Bock et al., *Methods for multidimensional event classification: a case study using images from a Cherenkov gamma-ray telescope*. Nuclear Instruments and Methods A, 516(2–3):511–528, 2004.
- UCI Machine Learning Repository. *MAGIC Gamma Telescope Data Set*. Dataset ID: 159. Available at `https://archive.ics.uci.edu/dataset/159/magic+gamma+telescope`.
- F. Maggioni, *Optimization* (course materials). Università degli Studi di Bergamo, A.Y. 2024/2025.
- S. Sra, S. Nowozin, S. J. Wright (eds.), *Optimization for Machine Learning*. MIT Press, 2012.

Thank you!